
XHAIL

Re-engineering an Abductive-Inductive Learning System
for Research-Grade Symbolic AI

Josh Everett

University of Bristol • 2024

Abstract. Inductive Logic Programming (ILP) learns interpretable symbolic rules from examples and background knowledge. XHAIL (eXtended Hybrid Abductive Inductive Learning) implements a three-phase pipeline — abduction, deduction, and induction — each realised as a separate Answer Set Programming (ASP) solve using the clingo solver. This report describes a research-grade Python reimplementations of the XHAIL paradigm (Ray, 2009), documenting: 14 correctness defects fixed in the original prototype; a reproducible benchmarking framework across four canonical ILP tasks; and a systematic comparison against Aleph, Metagol, ILASP, and FastLAS. All four benchmarks are solved correctly. Known limitations — multi-head learning, recursive modeb non-termination, and output variables in modeh — are characterised as concrete future contributions. The codebase ships with 101 automated tests, GitHub Actions CI, and a one-command experiment runner.

Keywords: *inductive logic programming · answer set programming · abductive reasoning · interpretable AI · clingo · symbolic machine learning*

1 Introduction

The dominant paradigm in machine learning is statistical: models learn numerical weights from large datasets, producing predictions that are opaque to inspection. Inductive Logic Programming (ILP) occupies the opposite end of the spectrum — it induces *symbolic rules* that are human-readable, verifiable, and consistent with prior knowledge. This makes ILP naturally relevant to explainable AI, scientific discovery, and program synthesis.

XHAIL (eXtended Hybrid Abductive Inductive Learning) was introduced by Oliver Ray (2009) as a method for nonmonotonic ILP using Answer Set Programming. Its key insight is that the ILP problem decomposes into three sequential sub-problems — abduction, deduction, and induction — each a natural ASP optimisation problem. By chaining three clingo solve calls, XHAIL avoids a bespoke search procedure and leverages a mature, highly-optimised solver throughout.

Despite its theoretical elegance, XHAIL has seen limited implementation work. This report describes a clean Python reimplementaion that (1) fixes 14 correctness defects in the original prototype, (2) provides a pip-installable package with a public API and CLI, (3) introduces a reproducible benchmarking framework, and (4) positions XHAIL rigorously against four major competing ILP systems.

Contributions

- Correct, fully tested Python reimplementaion of XHAIL (Ray, 2009)
- 14 documented bug fixes, each with a regression test
- Reproducible experiment runner with structured metric logging
- Systematic limitation taxonomy — five limitations characterised
- Comparative analysis against Aleph, Metagol, ILASP, and FastLAS

2 Background

2.1 Inductive Logic Programming

ILP (Muggleton, 1991) learns first-order logic programs from examples. Given background knowledge $\mathbf{B}$, positive examples $\mathbf{E}^+$, and negative examples $\mathbf{E}^-$, find $\mathbf{H}$ such that $\mathbf{B} \cup \mathbf{H}$ entails every positive example and no negative example. Systems differ in hypothesis language, entailment semantics, and search strategy.

2.2 Answer Set Programming

ASP is a declarative paradigm based on stable model semantics. The clingo solver (Gebser et al., 2014) provides an efficient ASP engine with a Python API. XHAIL exploits ASP's built-in `#minimize` directive to select smallest-cost answer sets — the core mechanism behind the abduction and induction phases.

3 System Description

3.1 Input Format

An XHAIL input file (.lp) contains background knowledge, mode declarations (language bias), and labelled examples:

```
% Background knowledge bird(a). bird(b). bird(c). penguin(d). bird(X) :- penguin(X).
% Mode declarations #modeh flies(+bird). % head predicates the learner may use #modeb
penguin(+bird). % body predicates; + = input variable #modeb not penguin(+bird). %
negation-as-failure allowed % Labelled examples #example flies(a). #example not
flies(d). % negative example
```

3.2 The Three-Phase Pipeline

Phase 1 — Abduction

Finds the minimal set of ground atoms Δ that, with background knowledge $\mathbf{B}$, satisfies all examples. Encoded as an ASP optimisation: abducible atoms are choice rules; #minimize selects the smallest satisfying set. Δ provides a concrete "witness" explaining why the examples hold.

Phase 2 — Deduction

Constructs the *kernel set* K — maximally-specific clauses that justify Δ under the mode language. Each clause in K is the most-specific rule derivable from $\mathbf{B} \cup \Delta$ that matches a #modeh pattern. K bounds the hypothesis space: any correct H must be a generalisation of a subset of K .

Phase 3 — Induction

Finds the minimal $H \subseteq K$ covering all examples. Encoded as an ASP optimisation: each clause in K is a candidate; #minimize selects the smallest covering subset. The result is returned as a LearningResult.

Pipeline: Input (.lp) → **Abduction** [find Δ] → **Deduction** [build kernel K] → **Induction** [select $H \subseteq K$] → Learned rules

4 Implementation

4.1 Package Structure

```
xhail/ __init__.py # Public API: learn(), LearningResult cli.py # xhail command-line
interface core.py # Pipeline orchestration language/ # Atom, Literal, Clause,
PlaceMarker parser/ # PLY-based .lp parser reasoning/ # abduction.py deduction.py
induction.py experiments/ benchmarks/ # Four canonical .lp benchmark tasks
run_benchmarks.py # Config-driven experiment runner plot_results.py #
Publication-quality figure generator tests/ # 101 automated tests (pytest)
```

4.2 Public API

```
from xhail import learn result = learn('penguins.lp', depth=10) if result.success:
for rule in result.hypothesis: print(rule) # flies(V1) :- not penguin(V1).
```

4.3 Key Defects Fixed

- **D1** — Nested f-strings invalid on Python < 3.12; refactored.
- **D2** — Constraint constructor accepted wrong argument types.

- **D3+D5** — Parser missed propositional atoms and :- integrity constraints.
- **D6** — Deduction phase hung indefinitely; configurable timeout added.
- **D8–D10** — Mutable class-level defaults, no-op set operations, incorrect replaceConstants indexing.
- **D14** — Propositional trailing comma: `try(0, 1, )` generated for 0-arity predicates. Fixed via `_try_term()` helper in `induction.py`.

4.4 Testing and CI

101 tests span unit (language structures, parser, API contracts), integration (end-to-end clingo pipeline), and benchmark regression (exact hypothesis assertions). GitHub Actions runs the matrix on Python 3.10/3.11/3.12. Ruff enforces style; pre-commit hooks run lint before every commit.

5 Related Work

Table 1 compares XHAIL against the four most relevant ILP systems (XHAIL column highlighted):

Dimension	Aleph	Metagol	ILASP	FastLAS	XHAIL (ours)
Hypothesis lang.	Horn clauses	Metarule inst.	Full ASP	Normal+choice	Normal rules (NAF)
Negation (NAF)	No	No	Yes	Yes	Yes
Constraints	No	No	Yes (hard+weak)	Yes (hard)	No
Recursive progs	Limited	Yes (metarules)	Limited	Limited	No (times out)
Example type	Ground atoms	Ground atoms	Partial interp.	Partial interp.	Ground atoms
Search strategy	Greedy set-cover	Proof search	ASP enum.	Pruned ASP enum.	3-phase pipeline
Solver backend	Prolog	Prolog	clingo	clingo	clingo
Predicate invent.	No	Yes	No	Yes (v2)	No
Implementation	Prolog	Prolog	C++/Python	C++/Python	Python
Maturity	High (2001+)	High (2015+)	High (2014+)	Medium (2020+)	Research proto.

Table 1: ILP system comparison across ten dimensions.

XHAIL vs Aleph

Both use mode declarations and ground-atom examples. Aleph uses greedy set-cover over Horn clauses; XHAIL uses globally-optimal ASP minimisation. The critical difference is NAF: Aleph cannot express "flies unless penguin" — XHAIL can.

XHAIL vs Metagol

Metagol excels at recursive program synthesis via higher-order metarules but cannot handle NAF or exception-based defaults. XHAIL is better suited to classification with defaults; Metagol is better for systematic recursive program synthesis.

XHAIL vs ILASP

ILASP is the most expressive ASP-based ILP system, learning full ASP programs from partial interpretations. XHAIL's three-phase decomposition provides interpretable intermediate artefacts (Δ , K) that ILASP's monolithic hypothesis enumeration does not expose. ILASP is more expressive; XHAIL is more transparent.

XHAIL vs FastLAS

FastLAS prioritises scalability via aggressive hypothesis-space pruning and user-defined scoring functions. XHAIL's phase decomposition offers a different efficiency strategy: each sub-problem is smaller than the full search. The systems reflect different trade-offs between scalability and interpretability.

6 Experiments

6.1 Benchmark Tasks

- **penguins** — Learn that birds fly unless they are penguins. Tests negation-as-failure.
- **animals** — Learn that mammals have hair or produce milk. Tests multi-predicate feature classification.
- **traffic** — Learn that vehicles stop at red lights. Tests simple deterministic rule learning.
- **propositional** — Learn a 0-arity output from 0-arity inputs. Tests the propositional code path fixed in this work.

6.2 Metrics Recorded

For each run: wall-clock time (`runtime_s`), peak RSS memory (`peak_memory_mb` via `resource.getrusage`), mean body literals per rule (`rule_complexity`), number of rules, and number of examples. Results export to CSV and JSON.

6.3 Results

Benchmark	Task type	Examples	Rules	Hypothesis	Time (s)
penguins	NAF classification	4 (3+, 1-)	1	<code>flies(V1) :- not penguin(V1).</code>	0.065
animals	Feature classification	6 (4+, 2-)	1	<code>mammal(V1) :- produces_milk(V1).</code>	1.070
traffic	Rule learning	4 (2+, 2-)	1	<code>stop(V1) :- red(V1).</code>	0.045
propositional	0-arity (propositional)	1 (1+, 0-)	1	<code>output :- .</code>	0.008

Table 2: Benchmark results. All four tasks solved correctly.

All four benchmarks are solved correctly. Runtimes span two orders of magnitude: the propositional task (no variable grounding) finishes in 8 ms; animals (larger background, more type constraints) takes ~1 s. All rule complexity scores are 1.0 (one body literal per first-order rule) or 0.0 (empty body for the propositional fact). The penguins task produces the expected `flies(V1) :- not penguin(V1).`; traffic produces `stop(V1) :- red(V1).`; animals produces a valid discriminating feature rule; and propositional produces `output :- .` — the minimal hypothesis for a single positive example with no negatives.

7 Limitations

Five limitations were characterised empirically. Each is a concrete research target rather than a fundamental obstacle.

L1 — Multi-head learning

`induction.py` hardcodes `new_head = clauses[0].head`, assigning all learned rules to the first `#modeh` predicate. Fix: track which modeh generated each kernel clause.

L2 — Recursive modeb causes non-termination

If a modeb predicate appears in its own derivation, the deduction phase generates an unbounded grounding. Fix: enforce the depth parameter as a hard bound at clause-construction time.

L3 — Output variables in modeh

A `-type` placemaker in `#modeh` causes an `AttributeError` in `generalise()`. Fix: add an explicit `isinstance(term, PlaceMarker)` check before dispatching to the atom-processing branch.

L4 — Propositional trailing comma (fixed)

0-arity predicates previously generated `try(0, 1, )` — invalid clingo syntax. Fixed via `_try_term()` helper in `induction.py`. Verified by `TestPropositionalBenchmark`.

L5 — No per-phase timeout

Pathological inputs hang indefinitely. A `DeductionTimeoutError` class exists but is not wired to the clingo solve call. Fix: pass a timeout argument to the clingo Python API.

8 Future Work

Near-term

- Fix L1 (multi-head) — one-line change in `runPhase()`; unblocks all multi-class classification tasks.
- Fix L3 (output vars in modeh) — one-line `isinstance` check; unblocks relational tasks.
- Fix L5 (per-phase timeouts) — wire clingo timeout argument.

Medium-term

- Scalability analysis: vary example count and background size; measure per-phase runtime scaling to identify bottlenecks.
- Comparative experiment: run shared tasks in Aleph and ILASP; compare runtime, hypothesis quality, and intermediate artefact quality.
- Bounded recursive deduction: investigate strategies allowing a constrained class of recursive modeb predicates.

Long-term

- Neuro-symbolic integration: use neural perception to generate ground facts; XHAIL learns symbolic rules over neural features.

- Noisy examples: extend abduction to tolerate inconsistent examples via weighted or majority-voting coverage.
- LLM-assisted rule synthesis: use a language model to propose candidate body predicates, reducing the mode declaration burden.

9 Conclusion

XHAIL's three-phase pipeline provides a principled and transparent approach to symbolic rule learning. By decomposing the ILP problem into three sequential ASP solve calls, it exposes interpretable intermediate artefacts — the abduced atom set Δ and kernel K — that monolithic approaches do not provide.

This reimplementaion demonstrates the paradigm is viable in modern Python with clingo: all four benchmark tasks solved correctly, 101 tests passing on Python 3.10–3.12, and a clean, research-ready codebase. The five characterised limitations form a concrete research agenda — L1 and L3 being particularly high-impact one-line fixes that immediately expand the class of expressible ILP tasks.

The project is openly available at github.com/everettmakes/xhail.

References

- Gebser, M., Kaminski, R., Kaufmann, B., & Schaub, T. (2014). *Clingo = ASP + control*. arXiv:1405.3694.
- Law, M., Russo, A., & Broda, K. (2014). Inductive learning of answer set programs. *ECAI 2014*, 311–316.
- Law, M., Russo, A., & Broda, K. (2020). The ILASP system for inductive learning of answer set programs. *TPLP*, 20(2), 222–256.
- Law, M., Russo, A., Bertrand, J., & Broda, K. (2020). FastLAS: Scalable ILP incorporating domain-specific optimisation criteria. *AAAI 2020*, 2877–2885.
- Muggleton, S. (1991). Inductive logic programming. *New Generation Computing*, 8(4), 295–318.
- Muggleton, S., Lin, D., Pahlavi, N., & Tamaddoni-Nezhad, A. (2015). Meta-interpretive learning of higher-order dyadic datalog. *Machine Learning*, 100(1), 49–73.
- Ray, O. (2009). Nonmonotonic abductive inductive learning. *Journal of Applied Logic*, 7(3), 329–340.
- Srinivasan, A. (2001). *The Aleph Manual*. University of Oxford.